

Master Tester Agent

Core Platform & Traceability Dashboard

MTA · Requirement-Traceable Test Orchestration · Multi-Surface AI & Software Evaluation

Document ID	BRD-MTA-001
Version	1.0
Date	6 August 2026
Author	Priyansh Balwani — [team/function]
Status	Draft — pending stakeholder review
Scope of this document	Phase 0–3 delivery: MTA core platform, traceability dashboard, and first-wave adapters (Web/UI, API, AI agent)
Target platforms	Internal web application (registry + dashboard); CI/CD integration (GitHub Actions/Jenkins); adapters against web, API, and agent solutions
Reference	RESEARCH_Industry_Landscape.md (industry research underpinning this design); supersedes BRD_Master_Agent_Tester.md v0.1 and BRD_Master_Tester_Agent_v2.md — those are retained for history, not for use

Top Five Features

#	Feature	The question it answers	Section
1	Central solution registry & live traceability matrix	Is every requirement in this solution's BRD/TSD actually covered by a test, and did it pass?	5.1
2	One platform, many solution types	Can we test an AI agent, an API, and a web app without three disconnected tools?	5.2, 6
3	AI-specific evaluation (hallucination, prompt injection, tool-calling, reasoning)	Is this agent safe and grounded, not just functionally correct?	5.3, 6.3
4	Confidence-scored, human-escalated verdicts	Can we trust an automated pass/fail, or does someone need to look at this one?	5.4, 7.3
5	CI/CD release gate	Can leadership see, at a glance, whether a release is compliant and safe to ship?	5.5, 9

Features 3 and the deeper AI-specific evaluators (RAG quality, memory validation, cross-platform consistency) depend on adapter/evaluator types not yet built in Phase 1 — see section 13 (Future Scope) for sequencing. Where a feature is listed here but its underlying adapter ships later, the dashboard shows it as “not yet covered” rather than a false pass.

1. Purpose

Across the organization, teams ship AI agents, RAG systems, APIs, internal tools, and client-facing web applications, each tested — if tested at all — by whoever built it, against whatever they personally thought to check. There is no shared record of whether a given release actually satisfies its own BRD/TSD, no consistent adversarial or edge-case coverage, and no single place leadership or a client-facing team can look to see whether a solution is release-ready.

This document scopes the first deliverable: the MTA core platform — a solution registry, a requirement-to-test traceability engine, an extensible adapter/evaluator framework, and a compliance dashboard — piloted against a small number of real solutions spanning different types, then extended adapter-by-adapter. It does not attempt to build every testing capability described in the long-term vision on day one; section 13 records what is deliberately deferred, and why.

2. Objectives

ID	Objective	Success measure
OBJ-01	Give every solution team and QA leadership a single source of truth for requirement coverage and release readiness	Traceability dashboard adopted as the release-gate reference by all pilot teams within 8 weeks of go-live
OBJ-02	Reduce manual QA effort per release	Measurable reduction in manual test-authoring/execution hours on pilot solutions
OBJ-03	Catch defects before clients or end users do	Defects found by MTA pre-release, tracked as a standing metric from go-live
OBJ-04	Make automated evaluation itself trustworthy	Judge/human agreement rate (Cohen's kappa) tracked and published per evaluation domain from first use
OBJ-05	Prove the adapter/evaluator architecture generalizes before over-investing	Two additional solution types onboarded post-pilot without a core-platform redesign
OBJ-06	Keep human review authoritative over anything automation can't confidently resolve	100% of sub-confidence-threshold verdicts routed to a human queue, none auto-resolved

3. Scope

3.1 Solution types in scope for Phase 1–3 (this delivery)

- AI agents exposed via a direct API/chat interface (web or programmatic)
- REST APIs / backend services with an OpenAPI specification
- Standalone web applications / internal dashboards

3.2 Solution types explicitly deferred (see section 13)

- RAG systems (retrieval-quality evaluator not yet built)
- Chat-platform bots — Teams, Slack (platform adapters not yet built)
- Enterprise integrations (queues, webhooks, ERP/SFTP)
- Long-horizon AI memory validation
- Cross-platform consistency checking (requires ≥ 2 platform adapters to exist first)

3.3 Testing types in scope for this delivery

- Functional, smoke, sanity, edge-case, negative/boundary, regression
- API schema/contract validation and fuzzing
- UI functional automation (visual regression deferred — see 13)
- LLM-as-judge evaluation of agent responses against a requirement/rubric
- Adversarial/security: prompt injection and jailbreak-resistance testing (static seed corpus only in Phase 1; adaptive/generative red-teaming is Phase 2+)

3.4 Testing types explicitly deferred (see section 13)

- Performance/load testing (including LLM-inference-specific latency/cost testing)
- Accessibility (WCAG) testing
- Business-rule/decision-table validation
- RAG retrieval-quality and memory-validation evaluation
- Visual regression testing

3.5 What this delivery deliberately excludes, permanently (not a phasing decision)

- Automatically remediating a failing solution — MTA reports defects; engineering teams fix them
- Replacing human sign-off on subjective quality (tone, brand voice, business acceptance) or on anything below MTA's confidence threshold
- Testing any solution not registered with MTA

Keeping the testing-type list disciplined for this delivery keeps the core traceability engine and dashboard the thing under scrutiny, rather than diluting review effort across nine adapter types built simultaneously.

4. Target Design — Dashboard & Deliverables

This is what a solution owner, QA lead, or executive actually sees. Priority: M = Must have, S = Should have, C = Could have.

4.1 Solution Registry View

ID	Requirement	Priority
BR-D-01	List every registered solution with type, owner, current version, and status	M
BR-D-02	Filter by solution type, owner, and status	M
BR-D-03	Show, per solution, links to its source BRD/TSD documents	M
BR-D-04	Flag solutions with no linked BRD/TSD as “unregistered requirements” rather than silently skipping them	M

4.2 Requirement Traceability Matrix View

ID	Requirement	Priority
BR-D-05	Per solution, show a live grid of requirement ID × latest test result, generated automatically from test authoring/execution metadata — never a manually maintained spreadsheet	M
BR-D-06	Drill down from any cell to the underlying test case, raw execution result, and evaluator rationale	M
BR-D-07	Flag requirements with zero mapped tests as an explicit coverage gap, not an implicit pass	M
BR-D-08	Tag each requirement/test with the ISO/IEC 25010 quality characteristic(s) it validates	S

4.3 Release Readiness / Compliance View

ID	Requirement	Priority
BR-D-09	Show pass/fail against a defined release threshold, per solution per version	M
BR-D-10	Show a single required-check-compatible status consumable by CI/CD, regardless of how many adapters/evaluators ran underneath	M
BR-D-11	Export a compliance report (PDF/CSV) per solution per version	S
BR-D-12	Show trend of pass rate across versions	S

4.4 Findings & Human Review Queue

ID	Requirement	Priority
BR-D-13	List open findings per solution with severity, confidence score, and evaluator rationale	M
BR-D-14	Route any verdict below the confidence threshold to this queue rather than auto-resolving it	M
BR-D-15	Let a reviewer approve, override, or annotate an auto-generated test case or requirement object before it enters the active suite	M

4.5 Judge Calibration Health View

ID	Requirement	Priority
BR-D-16	Show judge/human agreement rate (Cohen's kappa) per evaluation domain, trended over time	S
BR-D-17	Alert when agreement drops below a defined floor for a domain	S

Deferred to a later phase: ISO 25010 coverage radar, cross-platform consistency side-by-side view, RAG chunk-attribution drill-down, accessibility automation-confidence view, performance trend dashboards — see section 13.

5. Solution & Evaluation Requirements

5.1 Registry & Requirements Ingestion

ID	Requirement	Priority
BR-R-01	Register a solution of any in-scope type via an onboarding API/form	M
BR-R-02	Ingest BRD/TSD source documents (PDF, Word, Confluence/Notion export, or an OpenAPI spec)	M
BR-R-03	Extract structured requirement objects from prose using a multi-step (extract → classify → decompose) process, never single-shot	M
BR-R-04	Route every extracted requirement object through mandatory human review before it becomes authoritative	M
BR-R-05	Generate one or more traceable test cases per approved requirement, covering at minimum functional, edge-case, and negative/adversarial variants	M

5.2 Execution Adapters (Phase 1 wave)

ID	Requirement	Priority
BR-A-01	Common adapter protocol (connect / send_input / get_response / teardown) that any adapter, first-party or future third-party, implements identically	M
BR-A-02	Web/UI adapter built on a proven browser-automation engine, with auto-waiting and multi-browser support	M
BR-A-03	API adapter supporting OpenAPI-spec-driven schema validation and fuzz-case generation	M
BR-A-04	Agent adapter capable of single-turn and scripted multi-turn conversation exchange	M
BR-A-05	Adapter failures retry with backoff before being recorded as a hard failure	M

5.3 Evaluation Engine (Phase 1 wave)

ID	Requirement	Priority
----	-------------	----------

ID	Requirement	Priority
BR-E-01	Rule-based evaluator: schema validation, exact/regex match, latency SLA check	M
BR-E-02	LLM-as-judge evaluator, prompt-templated from the requirement and expected behavior	M
BR-E-03	Every evaluator emits a confidence/coverage signal alongside its pass/fail verdict	M
BR-E-04	Aggregation: a test passes only if every applicable evaluator passes; sub-threshold confidence routes to human review, never auto-resolves	M
BR-E-05	Static-corpus prompt-injection/jailbreak-resistance evaluator for agent adapters	M
BR-E-06	Adversarial/red-team corpus stored under access controls equivalent to an internal exploit database — separate repository, least-privilege access, audit logging	M

6. Solution Approach

6.1 Architecture

Figure 1 — MTA core platform. One registry, one requirements-to-test pipeline, one evaluation engine, one traceability store — feeding both a human dashboard and a machine-readable CI/CD status. Every adapter and evaluator speaks the same versioned protocol to the core, so the orchestrator never depends on adapter-specific internals.

FIGURE 1 — MTA CORE PLATFORM ARCHITECTURE

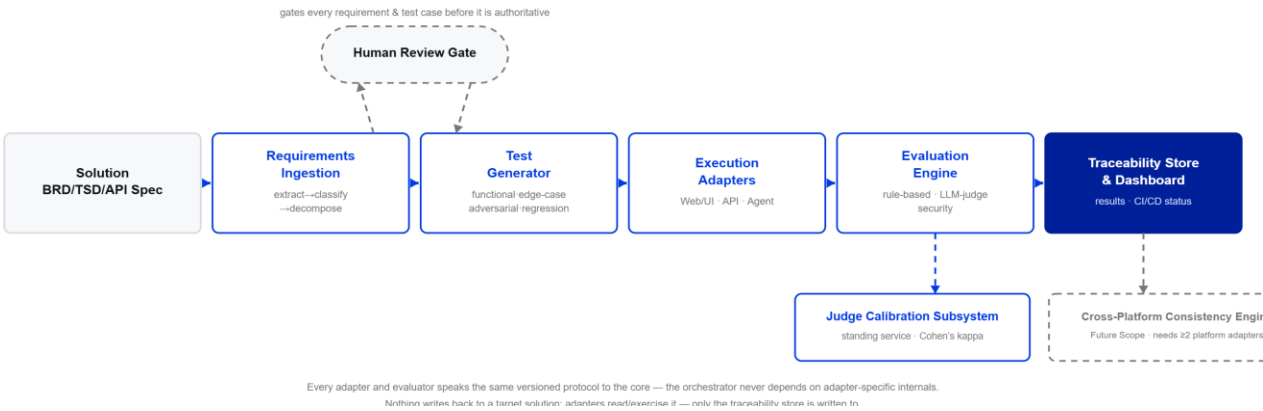


Figure 1 — MTA core platform architecture: registry → ingestion → test generation → execution adapters → evaluation → traceability & dashboard, with the human review gate and standing subsystems shown alongside the pipeline they govern.

Nothing in this pipeline writes back to a target solution. Adapters read/exercise the target; only the traceability store is written to.

6.2 Proposed Technology Stack

Proposed, to be confirmed with platform and security teams in Phase 0.

Layer	Proposal	Notes
-------	----------	-------

Layer	Proposal	Notes
Orchestration	Python, pytest-based core	Short-lived, stateless, parallel execution suits Phase 1's scope; a durable-workflow engine (Temporal) is deferred until a genuinely long-running human-in-the-loop campaign type exists
Web/UI adapter	Playwright	Enterprise-default automation engine; auto-waiting reduces flakiness versus Selenium/Cypress
API adapter	Schemathesis (spec-driven fuzzing)	Generates positive/negative/boundary cases directly from an OpenAPI spec rather than requiring hand-written cases
LLM services	Claude API — requirement extraction, LLM-as-judge, conversation simulation	
Tracing	OpenTelemetry / OpenInference	Vendor-neutral instrumentation, hedges against dependency on any single AI-eval vendor
Storage	Postgres — registry, requirements, test cases, results	
Dashboard	Internal web app	Or an extension of an existing internal BI/reporting tool — open question, see section 10
CI/CD	Hooks into existing pipeline (GitHub Actions/Jenkins) as a required check	
Secrets	Existing org secrets manager	
Adversarial corpus storage	Separate, access-controlled repository	Not the main application repo — see BR-E-06

6.3 Design Principles

- Requirement traceability is generated as a byproduct of test authoring and execution — never a manually maintained spreadsheet.
- No auto-generated requirement object or test case becomes authoritative without a human review step.
- Every evaluator emits a confidence signal; nothing below threshold is auto-resolved.
- Adapters and evaluators are added as plugins against a versioned protocol; the core never depends on adapter-specific internals, and MTA's own first-party adapters are held to the same public interface a future contributor would use.
- Mature, proven engines (Playwright, Schemathesis) are adopted rather than rebuilt — platform engineering effort goes into orchestration, traceability, and cross-domain normalization, not reimplementing browser automation or fuzzing engines.
- Live-testing-only is rejected as a substitute for a durable results store: every verdict is recorded immutably so release-readiness questions can be answered without re-running anything.

7. Non-Functional Requirements

Category	Requirement
Performance	A CI-gating subset of tests for a single solution completes within [X minutes]; full regression + adversarial suite completes within [X hours], run nightly or pre-release
Scalability	Architecture supports the Phase 1 pilot solution count without redesign, with a documented path to [N] solutions post-pilot
Reliability	Adapter-level retries with backoff before a hard failure is recorded (BR-A-05)
Security	Target-solution credentials stored in a secrets manager, never in plaintext config; adversarial corpus under exploit-database-equivalent access controls (BR-E-06)
Auditability	Test results are immutable once recorded; evaluator/judge version stored alongside every verdict
Extensibility	New adapters/evaluators added as plugins without modifying the core orchestrator (BR-A-01)

8. Risks

ID	Risk	Impact	Mitigation
R-01	Auto-generated requirement objects or test cases misinterpret ambiguous BRD/TSD language	High	Mandatory human review gate before anything becomes authoritative (BR-R-04)
R-02	LLM-as-judge grading is inconsistent between runs or drifts as models/prompts change	High	Judge-calibration subsystem with gold sets and tracked agreement rate from first use (OBJ-04, BR-D-16)
R-03	Pilot solution teams resist adopting another gate	Medium	Low-friction onboarding; demonstrate value via real defects caught in Phase 1
R-04	Adapter/evaluator protocol proves too rigid once a second solution type is added	High	Validate the protocol against a second, materially different solution type before declaring Phase 1 done (OBJ-05)
R-05	Confidence-threshold routing sends too much to human review, undermining the “reduce manual effort” objective	Medium	Track review-queue volume from week one; tune thresholds per domain rather than accept a fixed default
R-06	Leaked adversarial/prompt-injection corpus becomes a usable attack toolkit	High	Separate access-controlled repository, least-privilege grants, audit logging (BR-E-06)
R-07	Scope quietly expands back into deferred testing types (performance, accessibility, RAG) mid-delivery	Medium	Section 3.4/13 stand as agreed scope for this delivery; any earlier inclusion via formal change request

9. Delivery Phases

Phase	Content	Exit criteria
Phase 0 — Access & Discovery	Confirm pilot solution teams and their BRD/TSD availability/quality; confirm staging/sandbox access for each; confirm LLM model approval and cost budget; confirm hosting and stack	Pilot solutions and access confirmed; stack approved; go/no-go decision taken
Phase 1 — Core Platform	Registry; requirements ingestion with human-review gate; test generator; traceability & results store; dashboard (registry, matrix, findings, compliance views)	Traceability matrix reconciles against a hand-checked pilot solution; access-control tests pass; dashboard live
Phase 2 — Adapter Wave 1	Web/UI, API, and Agent adapters; rule-based, LLM-as-judge, and static-corpus security evaluators	All three pilot solutions onboarded and passing/failing correctly against known-good and known-broken versions
Phase 3 — CI/CD Gate	Tiered required-check integration; judge-calibration subsystem live with tracked agreement rate	Gate blocks a deliberately-broken build in a dry run; calibration dashboard shows a baseline agreement rate

Phases 2 and 3 both depend on a stable Phase 1 traceability store and should not start before Phase 1 exits.

10. Open Questions

- Where the registry and dashboard are hosted — a new service, or an extension of an existing internal BI/reporting tool
- Which LLM(s) are approved for the judge/extraction role, and the cost budget per test run
- Initial confidence-threshold values and gold-set size for the judge-calibration subsystem — to be set jointly with the QA guild during Phase 0
- Which two or three pilot solutions (by type) are volunteered for Phase 1–2

11. Future Scope — Phase 2+ (Deferred, Not Rejected)

Everything below is out of scope for this delivery and recorded here as a candidate for a later phase. Nothing here is committed, estimated, or approved; each item enters delivery only through its own scoping and sign-off.

#	Item	What it would unlock	What it would require
FS-01	RAG evaluator (retrieval/generation metrics, chunk attribution)	Testing RAG systems, not just agents/APIs/web apps	A RAG adapter capturing retrieved context alongside the answer
FS-02	Platform/bot adapters (Teams, Slack)	Testing chat-platform-deployed agents	Canonical activity model + per-platform translation layer

#	Item	What it would unlock	What it would require
FS-03	Cross-platform consistency engine	Verifying the same agent behaves equivalently across surfaces	At least two platform/web adapters live for the same solution (FS-02 dependency)
FS-04	Long-horizon AI memory validation	Testing multi-day/multi-session recall, knowledge updates, cross-session leakage	Purpose-built harness — no mature off-the-shelf toolchain exists
FS-05	Performance/load testing (incl. LLM-inference-specific)	Load/stress/soak testing for web, API, and LLM-backed solutions	A performance adapter (k6-based) plus a streaming-LLM sub-adapter
FS-06	Accessibility (WCAG) evaluator	Automated a11y scanning with human-review routing for non-automatable criteria	An axe-core-based adapter, confidence-tagged findings
FS-07	Business-rule/decision-table validation	Combinatorial test generation from decision tables	A PICT-style generator and a rule-versioned golden-set store
FS-08	Adaptive/generative red-teaming	Novel attack generation beyond a static corpus	PyRIT/Giskard-pattern adaptive attack orchestration
FS-09	Enterprise integration testing (queues, webhooks, ERP/SFTP)	Testing async and batch integration points	Testcontainers-backed broker adapters, contract/webhook stubs

A note on sequencing: FS-03 (cross-platform consistency) is the platform's clearest differentiator versus anything on the market, but it structurally cannot be built before FS-02, so it should not be read as a lower priority than it is — only as blocked.

12. Sign-Off

Role	Name	Signature	Date
Executive Sponsor			
QA/Testing Guild Lead			
Platform/Infra Lead			
Pilot Solution Owners (×2–3)			